Readme

Threadline, Group 01, Track 2. How to set it up and run it locally.

Code: https://github.com/apurwakumari16nus/Track2_Dispatcher

Demo link: TO BE ADDED BEFORE SUBMISSION

What you need

To do this	You need
Simulator, tests, offline evaluation	Python 3.11+, uv, Git. No API key
Web desk with live reading	The above, plus an Anthropic API key with credit
iPhone app	A Mac with Xcode 26 or newer and iOS 26. A simulator is enough. A real phone also needs your own Apple developer team
Telegram	A bot token from @BotFather and a group with Topics turned on
Live spoken words	`whisper-cli` from `whisper.cpp` and a small English model. Optional: without it, the phone's own words stand

No Java, database, Node.js or GPU is needed.

Set up

Mac or Linux:

```
git clone https://github.com/apurwakumari16nus/Track2_Dispatcher.git
cd Track2_Dispatcher
uv sync
cp .env.example .env
```

Windows PowerShell:

```
git clone https://github.com/apurwakumari16nus/Track2_Dispatcher.git
cd Track2_Dispatcher
uv sync
Copy-Item .env.example .env
```

For live reading, open `.env` and set these. Never commit this file.

```
LLM_PROVIDER=anthropic
ANTHROPIC_API_KEY=your-key
ANTHROPIC_INTAKE_MODEL=claude-sonnet-5-5
```

Run every command from the repository root. Keep `data/` beside `harness/`.

Check it works, with no API key

```
uv run pytest -q
uv run python eval/run_simulator.py --output-dir /tmp/threadline_sim
uv run python harness/simulate.py --shock --seeds 10
```

You should see 568 tests pass. `/tmp/threadline_sim/results.csv` should match `eval/results.csv` exactly.

Run the web desk

```
uv run streamlit run src/app.py
```

Open http://localhost:8501. Keep the date at **01/04/2026** and the objective at **Minimise lateness**, pick request R09, and evaluate it. You should get Nimble Needle, back 4 April, five days to spare, cost 180. **Confirm** writes one line to the local log. Nothing goes to any workshop.

With no API key, open **System details** in the sidebar and choose **Rehearsal: test fixtures**. These are labelled synthetic extractions, not model output.

Run the iPhone app in a simulator

Start the desk in one terminal:

```
uv run python src/api.py
```

Then open `ios/Threadline.xcodeproj` in Xcode, pick an iPhone simulator and press Run. The app connects to `http://127.0.0.1:8600`. The desk starts in demo mode, which writes only to rehearsal logs and begins from eight earlier decisions. Without an API key it reads from test fixtures. In the simulator, speech is a labelled sample sentence, because there is no dependable microphone.

If you change `ios/project.yml`, run `xcodegen generate` in `ios/`.

On a real iPhone

1. Set your own team in Xcode under Signing and Capabilities. The committed project uses ours.
2. Put the Mac and phone on one network. The phone's hotspot is the most reliable.
3. Run the desk on the network. It needs a token so nobody else can use it:

```
THREADLINE_API_TOKEN=pick-a-token uv run python src/api.py --lan
```

4. Run the app from Xcode. In Settings, enter `http://<your Mac's name>.local:8600` and the token.

Lock Screen updates with the app closed need an Apple push key (`ios/AuthKey_XXXXXXXXXX.p8`), which is never committed. Without one, the Island catches up when you open the app.

Connect Telegram (optional)

1. Set `TELEGRAM_BOT_TOKEN` in `.env`, and make one group with Topics turned on. Add the bot.
2. Send `/chatid`, and add the ID to `TELEGRAM_ALLOWED_CHATS`.
3. In a topic per workshop or client, send `/iam QuickStitch` or `/iam Cotton Club`. Send `/iam Team` in the team's topic.

`src/api.py` runs the bot itself, so do not also run `src/telegram_bot.py`. This link-up passes automated tests with a stand-in and has not been tried in a real group.

Reproduce the evaluation

No API key is needed for any of these. They re-score recorded model replies or use no model.

```
uv run python eval/replay_cache.py eval/guardrails_v2/sonnet_dev/inbox_audit.json /tmp/cache_dev
uv run python eval/run_inbox.py --offline --cache-dir /tmp/cache_dev --model claude-sonnet-5-5 --labels eval/labels_v2.csv --output-dir /tmp/dev
uv run python eval/replay_cache.py eval/guardrails_v2/sonnet_heldout/inbox_audit.json /tmp/cache_held
uv run python eval/run_inbox.py --offline --cache-dir /tmp/cache_held --model claude-sonnet-5-5 --labels eval/labels_heldout_v2.csv --requests data/heldout_requests.txt --output-dir /tmp/held
uv run python eval/run_redteam.py --offline --output-dir /tmp/red
uv run python eval/run_rule_baseline.py
uv run python eval/run_explanation_review.py eval/guardrails_v2/sonnet_dev/inbox_audit.json eval/guardrails_v2/sonnet_heldout/inbox_audit.json
```

Expected: 30/30 behaviour and 29/30 decision on the supplied set, 15/16 and 15/16 on held-out, 16/16 behaviour on the red team. The last two commands rewrite `eval/rule_baseline/` and `eval/explanation_review.csv`.

A live run (`uv run python eval/run_inbox.py`) makes 30 model calls and needs a key.

Where things are

Path	What it holds
`src/`	Desk: intake, guards, tools, allocator, control, API, Telegram, web UI
`ios/`	iPhone app, widgets, Siri intents
`harness/`, `data/`	Unchanged course simulator and data, plus held-out and red-team requests
`eval/`	Runners, labels, recorded results
`tests/`	Automated tests. No model calls by default
`docs/`	Design and specification notes. Start with `docs/mobile_app.md`

Troubleshooting

You see	Do this
`Language model not configured`	Add the key to `.env`, or choose a rehearsal source
The app cannot reach the desk	Start `src/api.py` first. On a phone, check the address, token and shared network
The phone app is slow	Xcode's scheme runs Release. Debug builds are much slower on a phone, and Low Power Mode slows it too
`Set THREADLINE_API_TOKEN first`	`--lan` needs a token, as above
Data file not found	Run from the repository root